Practical Worksheet 4

Vanilla GANs

DCGAN on MNIST & CIFAR-10

Generative AI | Master's Course | University of Coimbra

Learning Focus

1. **Build a DCGAN**

   Convolutional Generator + Discriminator with BCE adversarial loss

2. **Train on Two Datasets**

   MNIST (resized to 32×32) and CIFAR-10 — same architecture, different complexity

3. **Checkpoints & Inference**

   Save/load model weights and generate new samples from a trained generator

4. **Latent-Space Interpolation**

   Walk between two latent vectors to inspect continuity and smoothness

Setup & Data Pipeline

Dependencies

PyTorch + torchvision

NumPy, Matplotlib, tqdm

Device auto-select: CUDA → MPS → CPU

Reproducibility

set_seed(42) — locks all RNGs

random, numpy, torch, cuda

Ensures comparable results across runs

Data Loaders: build_loaders()

MNIST — 1 channel, resized to 32×32, normalized to [-1, 1]

CIFAR-10

— 3 channels, 32×32 native, normalized to [-1, 1]

Train limit: 30k | Test limit: 5k | Batch size: 128

Generator Architecture

DCGenerator — ConvTranspose2d upsampling pipeline

$z \in \mathbb{R}^{100}$ (1×1) → **ngf×4** (4×4) → **ngf×2** (8×8) → **ngf** (16×16) → **C_out** (32×32)

Layer Pattern

```
ConvTranspose2d(in, out, 4, 2, 1)
BatchNorm2d(out)
ReLU(inplace=True)
```

Final layer: ConvTranspose2d → Tanh()

Key Details

Latent dim: 100 (standard DCGAN)

Base filters (ngf): 64

Output range: [-1, 1] via Tanh

No bias when using BatchNorm

z reshaped to (B, 100, 1, 1) before net

Discriminator Architecture

DCDiscriminator — strided Conv2d downsampling pipeline

Image 32×32 → **ndf** 16×16 → **ndf×2** 8×8 → **ndf×4** 4×4 → **σ(·)** 1×1

Layer Pattern

```
Conv2d(in, out, 4, 2, 1)
BatchNorm2d(out)
LeakyReLU(0.2, inplace=True)
```

First layer: no BatchNorm

Final: Conv2d → Sigmoid()

Key Details

Base filters (ndf): 64

Output: scalar probability ∈ [0, 1]

LeakyReLU slope: 0.2

Reshaped to (B, 1) at output

Mirrors generator structure exactly

Training Algorithm

Discriminator Update

```
1. opt_d.zero_grad()
2. d_real = BCE(D(real), ones)

3. z ~ N(0,I); fake = G(z)

4. d_fake = BCE(D(fake.detach()), zeros)

5. d_loss = d_real + d_fake

6. d_loss.backward(); opt_d.step()
```

Generator Update

```
1. opt_g.zero_grad()
2. z ~ N(0,I); fake = G(z)
3. g_loss = BCE(D(fake), ones)
4. g_loss.backward(); opt_g.step()
```

Warning: Don't forget .detach() on fake in D step! Without it, gradients flow into G.

Optimizer: Adam(lr=2e-4, betas=(0.5, 0.999)) | Loss: BCELoss | Labels: real=1, fake=0

Weight Initialization

DCGAN paper prescribes specific initialization — do not skip this step!

Convolutional Layers

```
init.normal_(weight, 0.0, 0.02)
```

Applies to both Conv2d and ConvTranspose2d layers

BatchNorm Layers

```
init.normal_(weight, 1.0, 0.02)
init.constant_(bias, 0)
```

Weight centered at 1.0, bias at zero

Usage: generator.apply(init_dcgan_weights) and discriminator.apply(init_dcgan_weights)

The .apply() method walks every submodule and calls init_dcgan_weights on each one

Part 1: MNIST DCGAN

32×32	1 ch	30k	20
Image size	Grayscale	Train images	Epochs

Your TODO Steps

1. Instantiate DCGenerator(latent_dim=100, image_channels=1) and DCDiscriminator(image_channels=1)

2. Apply init_dcgan_weights to both models

3. Run train_gan() with the MNIST loader — observe loss curves

4. Save checkpoint with save_checkpoint() for later inference

Part 2: CIFAR-10 DCGAN

32×32	3 ch	30k	30
Image size	RGB color	Train images	Epochs

Same Pipeline, New Challenge

Repeat the MNIST workflow with `image_channels=3`

CIFAR-10 is harder: 10 diverse object classes instead of uniform digits

More epochs needed (30 vs 20) to reach acceptable quality

Expect noisier/blurrier samples — this is a known DCGAN limitation at 32×32 RGB

Inference & Latent Walk

run_inference()

1. Set seed for reproducibility
2. Sample z from N(0, I), shape (n, 100)
3. Generate: fake = G(z)
4. Display 5x5 grid of generated images

All in @torch.no_grad() context

latent_walk()

1. Sample two endpoints: z0, z1
2. Interpolate: z(a) = (1-a)*z0 + a*z1
3. Generate images for a in [0, 1]
4. Visualize smooth morphing

Tests if latent space is continuous

Checkpoint Loading

```
gen, cfg, hist = load_dcgan_generator_for_inference(path)
```

Loads state_dict + config (latent_dim, channels, image_size). Sets eval mode.

Common Pitfalls

Missing .detach() on fake in D step	Without it, gradients flow into G — D update accidentally trains G
Reusing old fake tensor in G step	Sample fresh z and generate new fakes after D optimizer step
Wrong labels for G loss	Generator uses real_targets (=1), NOT fake_targets — it wants D to believe fakes are real
Shape mismatch in BCE	Ensure D output and targets are both (B, 1) — watch for squeeze/view issues
Skipping weight initialization	DCGAN is sensitive — random PyTorch init often leads to mode collapse early

Tip: if D loss drops to 0 quickly, the generator has likely collapsed — restart with different seed

Reflection Questions

1. What differences did you observe between MNIST and CIFAR-10 DCGAN stability?
2. Did weight initialization and training length change sample quality significantly?
3. Which visual cue best indicates early mode collapse in your generated grids?
4. What is the first single hyperparameter you would tune next, and why?

Write your answers in the final notebook cell